torch.cuda.make_graphed_callables#

https://pytorch.org/docs/stable/generated/torch.cuda.make_graphed_callables.

Description:

Accept callables (functions or nn.Modules) and returns graphed versions. Each graphed callable's forward pass runs its source callable's forward CUDA work as a CUDA graph inside a single autograd node. The graphed callable's forward pass also appends a backward node to the autograd graph. During backward, this node runs the callable's backward work as a CUDA graph. Therefore, each graphed callable should be a drop-in replacement for its source callable in an autograd-enabled training loop. See Partial-network capture for detailed use and constraints.

Function Signature

```
torch.cuda.make_graphed_callables(callables: Union[Module, Callable[[...], object]], sample_args: tuple[torch.Tensor, ...], num_warmup_iters: int = 3, allow_unused_input: bool = False, pool: Optional[_POOL_HANDLE] = None) → Union[Module, Callable[[...], object]][source]#

torch.cuda.make_graphed_callables(callables: tuple[Union[torch.nn.modules.module.Module, Callable[..., object]], ...], sample_args: tuple[tuple[torch.Tensor, ...], ...], num_warmup_iters: int = 3, allow_unused_input: bool = False, pool: Optional[_POOL_HANDLE] = None) → tuple[Union[torch.nn.modules.module.Module, Callable[..., object]], ...]

Parameters
```

Notes & Warnings

NOTE Note The requires_grad state of each Tensor in sample_args must match the state that's expected for the corresponding real input in the training loop.

⚠ WARNING Warning This API is in beta and may change in future releases.

⚠ WARNING Warning sample_args for each callable must contain only Tensors. Other types are not allowed.

⚠ WARNING Warning Returned callables do not support higher order differentiation (e.g., double backward).

⚠ WARNING Warning In any Module passed to make_graphed_callables(), only parameters may be trainable. Buffers must have requires_grad=False.

⚠ WARNING Warning After you pass a torch.nn.Module through make_graphed_callables(), you may not add or remove any of that Module's parameters or buffers.

⚠ WARNING Warning torch.nn.Modules passed to make_graphed_callables() must not have module hooks registered on them at the time they are passed. However, registering hooks on modules after passing them through make_graphed_callables() is allowed.

⚠ WARNING Warning When running a graphed callable, you must pass its arguments in the same order and format they appeared in that callable's sample_args.

> ⚠ WARNING
> Warning The automatic mixed precision is supported in make_graphed_callables() only with disabled caching. The context manager torch.cuda.amp.autocast() must have cache_enabled=False.

Detailed Documentation

Documentation

Accept callables (functions or nn.Modules) and returns graphed versions.

Each graphed callable's forward pass runs its source callable's forward CUDA work as a CUDA graph inside a single autograd node.

The graphed callable's forward pass also appends a backward node to the autograd graph. During backward, this node runs the callable's backward work as a CUDA graph.

Therefore, each graphed callable should be a drop-in replacement for its source callable in an autograd-enabled training loop.

See Partial-network capture for detailed use and constraints.

If you pass a tuple of several callables, their captures will use the same memory pool. See Graph memory management for when this is appropriate.

callables (torch.nn.Module or Python function, or tuple of these) – Callable or callables to graph. See Graph memory management for when passing a tuple of callables is appropriate. If you pass a tuple of callables, their order in the tuple must be the same order they’ll run in the live workload.

sample_args (tuple of Tensors, or tuple of tuples of Tensors) – Samples args for each callable. If a single callable was passed, sample_args must be a single tuple of argument Tensors. If a tuple of callables was passed, sample_args must be tuple of tuples of argument Tensors.

num_warmup_iters (int) – The number of warmup iterations. Currently, DataDistributedParallel needs 11 iterations for warm up. Default: 3.

allow_unused_input (bool) – If False, specifying inputs that were not used when computing outputs (and therefore their grad is always zero) is an error. Defaults to False.

pool (optional) – Token (returned by graph_pool_handle() or other_Graph_instance.pool()) that hints this graph may share memory with the indicated pool. See Graph memory management.

The requires_grad state of each Tensor in sample_args must match the state that’s expected for the corresponding real input in the training loop.

This API is in beta and may change in future releases.

sample_args for each callable must contain only Tensors. Other types are not allowed.

Returned callables do not support higher order differentiation (e.g., double backward).

In any Module passed to make_graphed_callables(), only parameters may be trainable. Buffers must have requires_grad=False.

After you pass a torch.nn.Module through make_graphed_callables(), you may not add or remove any of that Module’s parameters or buffers.

torch.nn.Modules passed to make_graphed_callables() must not have module hooks registered on them at the time they are passed. However, registering hooks on modules after passing them through make_graphed_callables() is allowed.

When running a graphed callable, you must pass its arguments in the same order and format they appeared in that callable's sample_args.

The automatic mixed precision is supported in make_graphed_callables() only with disabled caching. The context manager torch.cuda.amp.autocast() must have cache_enabled=False.
